

Test Plan

Clinic Booking System

Student Biruk Mulatu
Student ID [STUDENT ID — fill in before submission]
Instructor Abel Tadesse
Date 4 September 2026
Group size 1 (solo submission)

1. Scope

This plan covers the testing effort for the Clinic Booking System, a small Spring Boot web application built as the vehicle for this course's testing techniques. Per the assignment brief, the application itself earns no marks; this document plans the testing of its business logic and user interface, not the addition of further features.

1.1 In scope

- **Rule 1 — consultation fee by age** (`core.FeeCalculator`): three age bands (CHILD, ADULT, SENIOR) plus rejection of out-of-range ages.
- **Rule 2 — booking eligibility** (`core.BookingPolicy`): a three-condition decision table (slot free, no outstanding balance, ≥ 2 hours notice) evaluated in strict priority order.
- **Rule 3 — appointment lifecycle** (`core.AppointmentStateMachine`): the 5-state, 4-event state machine and its 15 invalid transitions.
- **Rule 3b — late cancellation fee**: a 24-hour guard condition on top of the CANCEL transition.
- The orchestration layer (`AppointmentService`) that wires the three rules to repositories, a `Clock`, and a notification service.
- The web layer (login, slot list, booking form, confirmation, my-appointments) and the one end-to-end user journey it supports.
- Continuous integration (GitHub Actions and Jenkins) and the regression it is required to catch.

1.2 Out of scope

- Non-functional testing: load, performance, accessibility, security penetration testing.
- Real SMS delivery — `NotificationService` has only a logging implementation, by design (CLAUDE.md); no real gateway exists to test against.
- Multi-user concurrency (two patients racing for the same slot at the same instant) — the eligibility check (`C1`) is tested against the repository state at request time, not under concurrent load.
- Formal user acceptance testing (UAT) with real clinic staff — there is no such audience for a course project; the Selenium system tests stand in for the acceptance level (see 2.1).

2. Approach

2.1 Levels and techniques

The suite follows the test pyramid: many fast unit tests, fewer integration tests, a small number of system tests. Each level uses the technique(s) it is best suited to:

Level	What it exercises	Formal techniques used
Unit	<code>core/</code> pure functions, in isolation; <code>AppointmentService</code> orchestration with all collaborators replaced by test doubles	Equivalence partitioning, boundary value analysis, decision table, state transition testing (Part B of this project)
Integration	<code>AppointmentService</code> + real Spring Data JPA repositories + real H2 database; controllers via <code>MockMvc</code> (real HTTP request/response cycle, no browser)	Confirms the wiring and the repository queries behind the rules above actually work, not just the boolean logic in isolation
System	The full booking journey through a real headless Chrome browser via Selenium, using the Page Object pattern	End-to-end path testing of the one required user journey: log in → view slots → book → see fee on confirmation → cancel

Acceptance-level testing is represented by the same Selenium journey: for a course project with no external stakeholder to run a separate UAT phase, the system test acting out the specified user journey (per the assignment's own example: "log in then place an order") is treated as the acceptance check that the application meets its use case.

2.2 Test doubles

The design deliberately builds in seams for all four test-double kinds the course requires:

Kind	Where	What it replaces
Stub	A <code>Clock.fixed(...)</code> injected into <code>AppointmentService</code> in unit tests	The system clock, so the 2-hour and 24-hour boundaries can be hit exactly instead of racing real time
Mock	Mockito <code>@Mock NotificationService</code> in <code>AppointmentServiceTest</code>	The SMS-sending collaborator, so a confirmation can be <code>verify()</code> -ed without sending one
Fake	The H2 in-memory database used by every <code>*IT</code> test	A production database — a real, working, but lightweight implementation
Spy	<code>@SpyBean NotificationService</code> in <code>AppointmentServiceIT</code>	Wraps the real <code>LoggingNotificationService</code> so calls can still be verified while the real logging behaviour still runs

2.3 Coverage target

At least 80% branch coverage of `et.aau.clinic.core`, enforced by a JaCoCo `check` rule so `mvn verify` fails the build if it regresses. See the coverage report for the actual figure achieved.

3. Entry and exit criteria

3.1 Entry criteria (before a test level is considered ready to run)

- Unit level: the corresponding `core/` class or service method compiles and its test-double seams (`Clock` , `NotificationService`) exist.
- Integration level: unit tests for the same functionality are green; the Spring context starts successfully under the `test` profile against H2.
- System level: the application starts on a random port under `@SpringBootTest` ; the pages under test have the stable element `id` attributes Selenium locators depend on.

3.2 Exit criteria (before the testing effort is considered complete)

- `mvn clean verify` passes: all unit, integration and system tests green, in one run.
- Branch coverage of `et.aau.clinic.core` is at least 80% (JaCoCo gate passing, not just reported).
- Every valid and invalid state transition in Rule 3's 5×4 grid has an explicit test.
- Every boundary listed in this project's domain rules (ages −1, 0, 17, 18, 64, 65, 120, 121; the 2-hour notice boundary; the 24-hour cancellation boundary) has an explicit test.
- GitHub Actions is green on the current `main` , and the Jenkins pipeline has been run at least once to a successful build via the provided Docker setup.
- A regression has been deliberately introduced, observed failing in CI, and fixed, with both runs recorded as evidence.
- All defects found during the effort are logged with a status of Closed, or are explicitly carried as residual risk in the test summary report.

4. Risk-based prioritisation

Priority is set by likely impact (mostly: does it produce a wrong fee, a wrong booking outcome, or a corrupted appointment state?) combined with how many conditions/branches the logic has to get right, since more branching means more ways to get it wrong.

Priority	Area	Why
P1 — highest	Rule 2, booking eligibility decision table (<code>BookingPolicy</code>)	Three conditions evaluated in a specific priority order; getting the order wrong silently produces the wrong rejection reason or, worse, an approval that should have been a rejection (e.g. double-booking a slot). Highest branch count of the three rules, and the one most directly tied to correctness of a scheduling system.

P1 — highest	Rule 1, fee boundaries (FeeCalculator)	Directly affects money charged to a patient. A one-line boundary slip (as demonstrated in the regression, see docs/regression-demo.md) silently over- or under-charges real people — exactly the kind of defect BVA exists to catch, and exactly what happened when it was deliberately introduced.
P2 — high	Rule 3, state machine invalid transitions	Data-integrity risk: an appointment that can be re-confirmed after cancellation, or attended twice, corrupts clinic records. 15 of 20 state/event pairs must be rejected, so the chance of missing one is real without a systematic (not hand-picked) test list.
P2 — high	Rule 3b, late cancellation fee boundary	A second, independent financial boundary (24 hours) layered on top of a valid CANCEL transition; easy to get an off-by-one on < vs ≤ here exactly as happened (deliberately) with Rule 1.
P3 — medium	End-to-end web journey (login → book → confirm → cancel)	Lower branch complexity than the rules above, but the only place a wiring mistake (wrong view name, wrong redirect, wrong model attribute) would actually surface to a user — the integration and system tests exist specifically to catch that class of bug.
P4 — lower	Static/read-only pages, login failure messaging	Few branches, low financial/data-integrity impact if imperfect; still covered, but not where test effort concentrates.

5. Schedule

Real dates for this solo project (compressed relative to the assignment's suggested three-week plan since the code phases were completed ahead of the documentation):

Date	Milestone
2026-08-30 (morning)	Phases 1–6: domain, core rules with unit tests, service layer, controllers, integration tests
2026-08-30 (afternoon)	Phases 7–8: Selenium system tests, GitHub Actions, Jenkinsfile
2026-08-30 (evening)	Deliberate regression demo run and caught in CI; DEF-002 found and fixed
2026-08-30 (late)	Jenkins pipeline verified end-to-end against the Docker setup; DEF-003 found and fixed
2026-09-04	This test plan and the remaining three PDF deliverables (test design, defect log + metrics, test summary + reflection) written up from the evidence above
By 2026-09-13	Submission deadline — repository link and four PDFs on Google Classroom

6. Roles

This is a solo submission (group size 1). One student carried out every role the assignment would otherwise distribute across a group of up to four:

Role	Responsibility	Owner
Developer	Application code: domain, core rules, service layer, web layer	Biruk Mulatu
Test designer	Deriving EP/BVA/decision-table/state-transition test cases (Part B)	Biruk Mulatu
Test automation engineer	Writing and maintaining the unit, integration and Selenium suites	Biruk Mulatu
CI/DevOps	GitHub Actions workflow, Jenkinsfile and Docker setup, regression demo	Biruk Mulatu
Defect/quality manager	Defect log, metrics, coverage reporting, this test plan	Biruk Mulatu